Quantum algorithms for numerical differentiation of expected values with respect to parameters

Koichi Miyamoto*
Center for Quantum Information and Quantum Biology, Osaka University
1-3 Machikaneyama, Toyonaka, Osaka, 560-8531, Japan
(Dated: November 23, 2021)

The quantum algorithms for Monte Carlo integration (QMCI), which are based on quantum amplitude estimation (QAE), speed up expected value calculation compared with classical counterparts, and have been widely investigated along with their applications to industrial problems such as financial derivative pricing. In this paper, we consider an expected value of a function of a stochastic variable and a real-valued parameter, and how to calculate derivatives of the expectation with respect to the parameter. This problem is related to calculating sensitivities of financial derivatives, and so of industrial importance. Based on QMCI and the general-order central difference formula for numerical differentiation, we propose two quantum methods for this problem, and evaluate their complexities. The first one, which we call the naive iteration method, simply calculates the formula by iterative computations and additions of the terms in it, and then estimates its expected value by QAE. The second one, which we name the sum-in-QAE method, performs the summation of the terms at the same time as the sum over the possible values of the stochastic variable in a single QAE. We see that, depending on the smoothness of the function and the number of qubits available, either of two methods is better than the other. In particular, when the function is nonsmooth or we want to save the qubit number, the sum-in-QAE method can be advantageous.

arXiv:2111.11016v1 [quant-ph] 22 Nov 2021

I. INTRODUCTION

Following the recent rapid development of quantum computing technologies, many researchers are now investigating applications of quantum algorithms to practical problems in industries. The quantum algorithms for Monte Carlo integration (MCI), which we hereafter abbreviate as QMCI, are representative examples [1–3]. Based on quantum amplitude estimation (QAE) [2, 4–13], they output an estimate on an expected value $E[F(S)]$ of a function F of a stochastic variable S with an error up to ϵ, making $O\left(\frac{1}{\epsilon}\right)$ calls to some oracles such as O_F, which calculates F, and O_S, which generate a quantum state corresponding to the probability distribution of S (see Section II C for the detail). This is often referred to as the *quadratic speedup* compared with classical counterparts, which have a query complexity of $O(\epsilon^{-2})$.

One of application targets of QMCI is finance, especially *financial derivative pricing* (for readers unfamiliar with this, we refer to [14] as a textbook). A financial derivative is a contract between two parties, in which amounts (payoffs) determined by some underlying asset prices are paid and received. Roughly speaking, a financial derivative price is given by an expected payoff under a given mathematical finance model for time evolution of underlying asset prices, and calculated typically by MCI in which asset price evolution is simulated. Large banks, which have large portfolios of financial derivatives, take a lot of time and cost for pricing calculation in daily business, and therefore QMCI is expected to provide a large benefit to financial industry through quantum speedup.

In this paper, we consider how to calculate derivatives[1] of an expected value with a parameter. Although there are some previous researches [15–17] on quantum algorithms to calculate derivatives of a given function, this paper is the first one focusing on quantum algorithms for derivatives of an expected value, as far as the author knows. For a function $F(S, x)$ of a stochastic variable S and a real number x, its expected value $V(x) := E[F(S, x)]$ with respect to the randomness of S for fixed x can be viewed as a function of x. We often calculate the derivatives of $V(x)$ such as $V'(x)$ and $V''(x)$. For example, for a financial derivative, a bank calculates not only its price but also the derivatives of the price with respect to input variables such as the present underlying asset price and model parameters. These are called *sensitivities* or *Greeks*, and have crucial roles for risk management in financial derivative business [14].

In many practical problems, when $V(x)$ does not have a closed formula, neither do its derivatives. Then, a naive way to calculate a derivative is some *finite difference* method such as *central difference*:

$$V'(x) \approx \frac{V(x+h) - V(x-h)}{2h} \tag{1}$$

where h is some positive real number. This is the lowest order approximation formula for the first derivative, and there are the higher order approximation formulas for higher order derivatives, which have more terms and use the values of V at a larger number of grid points on the x axis (see Section II B for the detail). Basically, taking smaller h leads to the higher accuracy. However, when V is calculated by some numerical method such as MCI, there is a subtlety. Namely, it is not appropriate to calculate $V(x+h)$ and $V(x-h)$ individually and then use (1). This is because the result of a numerical calculation accompanies an error. Suppose that we have erroneous estimates $\tilde{V}(x+h) = V(x+h) + \epsilon_1$ and $\tilde{V}(x-h) = V(x-h) + \epsilon_2$, where ϵ_1 and ϵ_2 are numerical errors and their absolute values are bounded by ϵ with high probability. Plugging these into (1) yields

$$\frac{V(x+h) - V(x-h)}{2h} + \frac{\epsilon_1 - \epsilon_2}{2h}. \tag{2}$$

* koichi.miyamoto@qiqb.osaka-u.ac.jp

[1] In this paper, when we simply say a *derivative*, it refers to a mathematical terms, that is, a derivative of a function. On the other hand, when we refer to a derivative as a financial product, we use a *financial derivative*.

Here, the second term is of $O\left(\frac{\epsilon}{h}\right)$. Divided by a small number h, this can be large even if the error level ϵ is suppressed well. In other word, to accurately calculate a derivative, we have to suppress an error tremendously so that the dividing factor h is compensated, which results in the large complexity.

In classical MCI, there exist some solutions to this issue [18]. For example, in a common way where we use a pseudo-random number sequence on behalf of random variables, using a same seed for $V(x+h)$ and $V(x-h)$ alleviates this difficulty, since ϵ_1 and ϵ_2 become close and cancel each other.

In the case of QMCI, how can we calculate derivatives? Contrary to classical MCI, it is difficult to cancel errors in different runs of QMCI, since they are independent by quantum nature. Instead, we consider the following way:

$$V'(x) \approx E\left[\frac{F(S, x+h) - F(S, x-h)}{2h}\right]. \tag{3}$$

That is, we calculate not the difference quotient of V but the expected value of the difference quotient of F with respect to x. There is only the error from one QMCI for (3), and therefore we do not have to concern the aforementioned issue.

However, there are other subtleties in this approach. First, it is possible that V is smooth but F is nonsmooth or even discontinuous. This often happens in financial derivative pricing, because nonsmooth payoff functions are ubiquitous in practice, as we will see in Section III A. In such a case, for some (S, x), the smaller h we take, the larger the difference quotient of F becomes. As explained in Section II C, in QMCI, we normalize the integrand so that it is in the interval $[0, 1]$ and encode it into the amplitude of a qubit. Since the complexity of QMCI becomes larger for the larger normalization factor, the nonsmoothness of F can lead to the large complexity. Second, even if F is smooth, taking small h for accuracy can cause the issue on the *qubit number* as follows. The smaller h is, the closer $F(S, x+h)$ and $F(S, x-h)$ are. Whether it is classical or quantum, a computer can perform only the finite precision computation, and, in order to avoid the cancellation of significant digits between $F(S, x+h)$ and $F(S, x-h)$, we have to calculate them with sufficiently high precision. This means that, for smaller h, we use the larger number of qubits to compute F. Although the similar issue on memory size exist also in classical MCI, the severity is higher in QMCI, since the large qubit overhead for error correction might largely limit the number of logical qubits available even in the future [19]. In particular, for problems like financial derivative pricing, where F is computed through the complicated procedure, this qubit issue is more serious.

Taking into account these points, this paper proposes two quantum methods for numerical differentiation of V, and evaluates their complexities in terms of the numbers of queries to the oracles such as O_F and O_S, focusing on their dependencies on the error tolerance ϵ. The first method, which we call the *naive iteration method*, simply calls O_F iteratively to calculate the finite difference formula for F term-by-term, and estimates its expected value by QAE. The second method, which we name the *sum-in-QAE method*, utilizes the quantum parallelism more deeply and is more nontrivial. That is, in this method, we perform the summation of terms in the finite difference formula *at the same time* as the sum over the possible values of S in one QMCI. As we will see below, when F is smooth and we can use so many qubits that we can take sufficiently small h, the naive iteration method is better in the aspect of query complexity, since we can use the lowest order difference formula. On the other hand, when F is nonsmooth, the sum-in-QAE method with the high order formula and large h is appropriate. Besides, even when F is smooth, if we save the qubit number as much as possible, the sum-in-QAE method can be more advantageous, depending on the parameter measuring the smoothness of F, which is introduced in Section III A.

The rest of this paper is organized as follows. In Section II is a preliminary one, which explains the notation we use in this paper, and gives brief reviews on numerical differentiation and QMCI. Section III is the main part of this paper, where we present the naive iteration method and the sum-in-QAE method, and evaluate and compare their complexities in the various situations on smoothness of F and qubit capacity. Section IV summarizes this paper.

II. PRELIMINARIES

A. Notation

$\mathbb{N}$ denotes the set of all positive integers, and $\mathbb{N}_0 := \mathbb{N} \cup \{0\}$ is the set of all non-negative integers. For every $x \in \mathbb{R}$, $\mathbb{N}_{\geq x} := \{i \in \mathbb{N} \mid i \geq x\}$ is the set of all positive integers not less than x. For every integer pair (m, n) satisfying $m \leq n$, we define $[m : n] := \{i \in \mathbb{Z} \mid m \leq i \leq n\}$, where $\mathbb{Z}$ is the set of all integers.

$\mathbb{R}_+$ is the set of all positive real numbers, and $\mathbb{R}_{\geq 0} := \mathbb{R}_+ \cup \{0\}$ is the set of all non-negative real numbers.

We denote the set of all k-combinations from a finite set E as $\mathcal{P}_k(E)$, where $k \in [|E|]$.

For given $x \in \mathbb{R}$ and $\epsilon \in \mathbb{R}_+$, we call any $y \in \mathbb{R}$ satisfying $|x - y| \leq \epsilon$ a ϵ-approximation of x.

For every $x \in \mathbb{R}$, $|x\rangle$ denotes a computational basis state on some quantum register, in which the bit string on the register corresponds to a finite precision binary representation of x.

B. Numerical differentiation

Now, let us briefly review numerical differentiation. It is the method to approximately calculate a derivative of a given real-valued function f on some interval on $\mathbb{R}$ in the case that we can calculate f but not its derivatives directly. Based on the definition that $f'(x) = \lim_{h\to 0} \frac{f(x+h)-f(x)}{h}$, we can approximate $f'(x)$ as

$$f'(x) = \frac{f(x+h) - f(x)}{h} + O(h), \tag{4}$$

using a sufficiently small positive real number h. This type of approximation is called the forward difference method. The

residual error term scales as $O(h)$ in this scheme. However, there is the *central difference method*, in which the error scales as $O(h^2)$, and it is therefore used more often:

$$f'(x) = \frac{f(x+h) - f(x-h)}{2h} + O(h^2). \tag{5}$$

Also in this paper, we consider this method. In fact, (5) is the lowest-order approximation in this way. There are the higher order approximations for higher order derivatives. The general order formula and its residual term were investigated in [20]. Here, we present them as the following theorem, which is same as Corollary 2.1 in [20] except some slight changes.

Theorem 1 (Corollary 2.1 in [20], modified)**.** *Let m, n be positive integers such that $m \leq 2n$. Let f be a function such that $f \in C^{2n+1}(\mathbb{R})$. Then, for any $x \in \mathbb{R}$ and $h \in \mathbb{R}_+$,*

$$\begin{aligned}
f^{(m)}(x) &= \mathcal{D}_{n,m,h}[f](x) + R_f(x,n,m,h)h^{2n-m+1} \\
\mathcal{D}_{n,m,h}[f](x) &:= \frac{1}{h^m}\sum_{j=-n}^{n} d_{n,j}^{(m)} f(x_j) \\
d_{n,j}^{(m)} &:= \begin{cases} \dfrac{(-1)^{m-j} m! a_{n,j}^{(m)}}{(n+j)!\,(n-j)!} & ;\ \text{for } j \in [-n:n]\setminus\{0\} \\ -\displaystyle\sum_{j\in[-n:n]\setminus\{0\}} d_{n,h,j}^{(m)} & ;\ \text{for } j = 0 \end{cases} \\
R_f(x,n,m,h) &:= \frac{(-1)^{m+1} m!}{(2n+1)!} \sum_{j\in[-n:n]\setminus\{0\}} \frac{(-1)^j f^{(2n+1)}(\xi_j) j^{2n+1} a_{n,j}^{(m)}}{(n+j)!\,(n-j)!} \\
a_{n,j}^{(m)} &:= \sum_{\substack{\{l_1,\dots,l_{2n-m}\}\in \\ \mathcal{P}_{2n-m}([-n:n]\setminus\{0,j\})}} \prod_{i=1}^{2n-m} l_i \quad \text{for } j \in [-n:n]\setminus\{0\}
\end{aligned} \tag{6}$$

holds, where, for every $j \in \{-n, -n+1, \dots, n\}$, $x_j := x + hj$ and ξ_j is some real number depending on x and x_j.

This theorem states that the central difference method using the values of f at $2n+1$ points with interval h outputs an approximation of $f^{(m)}$ with an error of $O(h^{2n-m+1})$.

Let us comment on a virtue of central difference in the case of odd m, which includes the formula (5) for the first derivative. In this case, $d_{n,0}^{(m)} = 0$ holds, as mentioned in [20]. In particular, for $n = \left\lceil \frac{m}{2} \right\rceil = \frac{m+1}{2}$, the minimum value of n, $d_{\frac{m+1}{2},0}^{(m)} = 0$ holds. That is, although (6) seemingly requires evaluating f at $2n+1 = m+2$ points, we actually need to evaluate f only at $m+1$ points, which is equal to the minimum number of points to calculate $f^{(m)}(x)$ by finite difference formulas. Nevertheless, the error is $O(h^2)$. This is contrasted with other methods such as forward difference method, which uses the $m+1$ values $f(x), f(x+h), \dots, f(x+mh)$ and gives an estimate of $f^{(m)}(x)$ with an error of $O(h)$.

Let us comment also on the domain of f. Originally, [20] presented Corollary 2.1 for a function f on some interval on $\mathbb{R}$. However, considering a bounded domain may make the discussion cumbersome, e.g., we must care whether the points in the numerical differentiation formula are within the domain. Therefore, for simplicity, we simply consider the case where f is defined and sufficiently smooth on $\mathbb{R}$ in this paper. Theorem 1 has been modified in such a way. We expect that extending the discussion to the case where the domain is bounded is possible with the essential parts not affected.

C. Quantum amplitude estimation and its application to Monte Carlo integration

We now present a brief explanation on QAE [2, 4–13]. Suppose that we want to solve the following problem: given an oracle A on a system of a quantum register R_1 and a qubit R_2 such that

$$A|0\rangle|0\rangle = \sqrt{a}|\psi_1\rangle|1\rangle + \sqrt{1-a}|\psi_0\rangle|0\rangle \tag{7}$$

with some $a \in (0,1)$ and some quantum states $|\psi_0\rangle$ and $|\psi_1\rangle$ on R_1, estimate a, the probability that we obtain 1 on R_2 in $A|0\rangle|0\rangle$, up to an error at most $\epsilon \in \mathbb{R}_+$. QAE is a quantum algorithm for this. Making $O\left(\frac{1}{\epsilon}\right)$ calls to A and $A^\dagger$, QAE outputs ϵ-approximation of a with a probability higher than a given value (say, 0.99).

There are some applications of QAE, and Monte Carlo integration is one of them [1–3]. Suppose that we want to calculate

$$E[F(S)] = \sum_{s\in\mathcal{S}} p_s F(s) \tag{8}$$

with an error up to $\epsilon \in \mathbb{R}_+$. Here, S is a stochastic variable which takes an element s in some finite set $\mathcal{S}$ with a probability $p_s \in [0,1]$, and F is a bounded real-valued function on $\mathcal{S}$, which satisfies $|F(s)| \leq C$ for any $s \in \mathcal{S}$ with some $C \in \mathbb{R}_+$. We assume the availability of the following oracles. The first one is O_S, which generates a quantum state corresponding to the distribution of S:

$$O_S|0\rangle = \sum_{s\in\mathcal{S}} \sqrt{p_s}|s\rangle. \tag{9}$$

Here, we assume that elements in $\mathcal{S}$ are associated with mutually different real numbers, and $|s\rangle$ denotes a computational basis state corresponding to the real number for $s \in \mathcal{S}$. Note that, in measuring this state, we obtain s with a probability p_s. The second one is O_F, which calculate $F(s,x)$ for every $(s,x) \in \mathcal{S}\times\mathbb{R}$:

$$O_F|s\rangle|0\rangle = |s\rangle|F(s)\rangle. \tag{10}$$

Using these oracles, we can perform the following operation on a three-register system, where the last one is an ancilla

qubit:

$$
\begin{aligned}
&|0\rangle|0\rangle|0\rangle \\
&\to \sum_{s\in\mathcal{S}} \sqrt{p_s}\,|s\rangle|0\rangle|0\rangle \\
&\to \sum_{s\in\mathcal{S}} \sqrt{p_s}\,|s\rangle|F(s)\rangle|0\rangle \\
&\to \sum_{s\in\mathcal{S}} \sqrt{p_s}\,|s\rangle|F(s)\rangle\left(\sqrt{\frac{1}{2}+\frac{F(s)}{2C}}\,|1\rangle+\sqrt{\frac{1}{2}-\frac{F(s)}{2C}}\,|0\rangle\right).
\end{aligned}
\tag{11}
$$

Here, we used O_S at the first arrow, O_F at the second arrow, and some arithmetic circuits [21–36] and a controlled rotation gate at the third arrow. Then, the probability to obtain 1 on the ancilla in the last state is

$$
P = \frac{1}{2} + \frac{1}{2C}\sum_{s\in\mathcal{S}} p_s F(s), \tag{12}
$$

which means that

$$
E[F(S)] = C(2P-1). \tag{13}
$$

Therefore, we obtain an approximation of $E[F(S)]$ by estimating P using QAE and calculating (13). If we want to estimate $E[F(S)]$ with an error up to ϵ, it is sufficient to estimate P with an error of $O\left(\frac{\epsilon}{C}\right)$. This means that the oracle Q, which corresponds to the operation (11), is called $O\left(\frac{C}{\epsilon}\right)$ times, and so are O_S and O_F, since Q contains one each of them.

Although we assume that the domain $\mathcal{S}$ of S is finite, we often consider unbounded and/or continuous stochastic variables, such as a normal random variable which can take any real number. Such a case can be boiled down to the above setup by a discrete approximation. That is, we can set lower and upper bounds for S and grid points between the bounds, and approximate S as a discrete stochastic variable taking any of the grid points. Actually, there are quantum algorithms for generating a quantum state like (9) which corresponds to a discretely approximated stochastic variable [38, 39]. Also note that, in quantum computation, the error from discrete approximation can be exponentially suppressed, which means that we can deal with 2^n grid points using n qubits. Hereafter, we consider that (8) covers the cases where $\mathcal{S}$ is not finite through the discrete approximation, and neglect such an approximation error.

III. QUANTUM ALGORITHM FOR NUMERICAL DIFFERENTIATION OF EXPECTED VALUES

A. Problem

Hereafter, we consider the following problem. As above, let S be a stochastic variable taking an element s in some finite set $\mathcal{S}$ with a probability $p_s \in [0,1]$. Suppose that we are given a map $F : \mathcal{S}\times\mathbb{R}\to\mathbb{R}$. We then define the expected value of F as

$$
V(x) := \sum_{s\in\mathcal{S}} p_s F(s,x) \tag{14}
$$

for every $x\in\mathbb{R}$, and regard this as a real-valued function of a real number x, which we call a *parameter*. Now, for given x, we want to calculate not only $V(x)$ but also $V^{(m)}(x)$, the m-th derivative of V at x, where m is a given positive integer.

In fact, this covers pricing and sensitivity calculation for financial derivatives. In these calculations, paths of time evolution of the underlying asset price are generated by some stochastic variables, and the financial derivative price is given as the expectation value of the payoff determined by the asset price. As a concrete example, let us consider the following problem. Consider a financial derivative written on the underlying asset whose price P_t at time t obeys the Black-Scholes model with a volatility σ and a risk-free rate r. At its maturity T, the payoff $f_{\text{pay}}(P_T)$ arises, where $f_{\text{pay}} : \mathbb{R}_+ \to \mathbb{R}$ is some function, and P_T, the asset price at T, is given by

$$
P_T = P_0 \exp\left(\sigma\sqrt{T}S + \left(r-\frac{1}{2}\sigma^2\right)T\right) \tag{15}
$$

with $P_0 \in \mathbb{R}_+$, the asset price at the present $t=0$, and a standard normal random variable S. Then, the present price of this contract is calculated as

$$
\begin{aligned}
&V(P_0,\sigma,r) = \\
&\quad \int_{-\infty}^{+\infty} ds\,\phi_{\text{SN}}(s) f_{\text{pay}}\left(P_0\exp\left(\sigma\sqrt{T}s + \left(r-\frac{1}{2}\sigma^2\right)T\right)\right),
\end{aligned}
\tag{16}
$$

where ϕ_{SN} is the density function of the standard normal distribution (see [14] for the details). We can make the price (16) correspond to (14), viewing any of P_0, σ, and r as x, and discretely approximating S. For example, if x is P_0, we can consider that $F(s,x) = f_{\text{pay}}\left(x\exp\left(\sigma\sqrt{T}s + \left(r-\frac{1}{2}\sigma^2\right)T\right)\right)$. This example is one of the simplest problems in practical pricing tasks, and banks often deal with more complicated contracts and use more advanced models. In such a case, the financial derivative price is often not expressed simply as (16), but calculated by some numerical method such as MCI, where many stochastic variables and parameters are involved.

For a bank, it is important to calculate not only a financial derivative price but also its derivatives. These are called *sensitivities* or *Greeks*, and have crucial roles in risk management. For example, in the above example, the following are representative ones: the first derivatives with respect to P_0, σ and r, which are called the *delta*, the *vega* and the *rho*, respectively, and the second derivatives with respect to P_0, which is called the *gamma*.

Then, how can we calculate $V^{(m)}(x)$, when V does not have the closed formula and neither do its derivatives? One way is the central difference method described in Sec. II B. That is, choosing $n \in \mathbb{N}_{\geq \frac{m}{2}}$ and $h\in\mathbb{R}_+$, and assuming that $V \in C^{2n+1}(\mathbb{R})$, we can approximate $V^{(m)}(x)$ by

$$
V^{(m)}(x) \approx \mathcal{D}_{n,m,h}[V](x) = \sum_{s\in\mathcal{S}} p_s \mathcal{D}_{n,m,h}[F(s,\cdot)](x), \tag{17}
$$

where

$$\mathcal{D}_{n,m,h}[F(s,\cdot)](x) = \frac{1}{h^m}\sum_{j=-n}^{n} d_{n,j}^{(m)} F(s, x + jh). \tag{18}$$

Now, we have the following questions:

- can we construct a quantum algorithm to calculate the above numerical differentiation?
- what is the best setting, e.g. n and h, to reduce complexity keeping the desired accuracy?

For such a quantum algorithm, it is plausible to assume the availability of the following oracles. The first one is O_S mentioned above, which performs the operation (9). The second one is O_F, which is similar to (10), but now calculate $F(s,x)$ for every $(s,x) \in \mathcal{S} \times \mathbb{R}$:

$$O_F \left|s\right\rangle \left|x\right\rangle \left|0\right\rangle = \left|s\right\rangle \left|x\right\rangle \left|F(s,x)\right\rangle. \tag{19}$$

In the context of financial derivative pricing, O_F generates an asset price path and computes a payoff. Hereafter, we consider that O_F consumes a longer time and more qubits than O_S. This is the case in some practical problems such as financial derivative pricing under an advanced model, which is the very target of quantum speedup, since the asset price evolution is based on some complicated stochastic differential equation. In particular, in the QMCI method proposed in [40], in which, for the sake of qubit reduction, we calculate the integrand using pseudo-random numbers (PRNs) sequentially generated on a single register, O_S corresponds to generation of an equiprobable superposition of integer indexes which specify the start point of a PRN sequence and is implemented just by a set of Hadamard gates, whereas O_F contains complicated calculations such as asset price evolution and PRN generation. We hence focus on O_F in the discussion on query complexity and qubit number in the quantum algorithms proposed later. Practically, it is more plausible to consider the following oracle $O_{F,\epsilon}$ rather than O_F. $O_{F,\epsilon}$ performs the operation

$$O_{F,\epsilon} : \left|s\right\rangle \left|x\right\rangle \left|0\right\rangle \mapsto \left|s\right\rangle \left|x\right\rangle \left|F_\epsilon(s,x)\right\rangle \tag{20}$$

for every $(s,x) \in \mathcal{S} \times \mathbb{R}$. Here, $\epsilon \in \mathbb{R}_+$ and $F_\epsilon : \mathcal{S} \times \mathbb{R} \to \mathbb{R}$ is a function such that

$$\forall (s,x) \in \mathcal{S} \times \mathbb{R}, |F_\epsilon(s,x) - F(s,x)| \le \epsilon, \tag{21}$$

and uses

$$O\left(\log^a\left(\frac{1}{\epsilon}\right)\right) \tag{22}$$

qubits including ancillas, where $a \in \mathbb{R}_+$. This reflects the fact that we can perform only finite precision computation on a quantum computer and better precision requires more qubits. For example, if we assume that calculation of F can be decomposed into arithmetic circuits [21–37], n-bit precision typically requires $O(n)$ qubits, which corresponds to (22) with $a = 1$. Of course, similar issues exist also for O_S and other circuits used in QMCI, but we hereafter consider this point only for O_F because of the assumption that it is most costly in terms of qubits.

For a quantitative discussion on accuracy and complexity of algorithms, we need some assumption on derivatives of V, which are our targets and affect the residual terms of the formula (6). In this paper, following [17], we consider functions with the following property, which are called Gevrey functions.

Definition 1. *Let $A, c \in \mathbb{R}_+$ and $\sigma \in \mathbb{R}$. The set of functions $f : \mathbb{R} \to \mathbb{R}$ such that*

$$|f^{(k)}(x)| \le A c^k (k!)^\sigma \tag{23}$$

for any $k \in \mathbb{N}_0$ and $x \in \mathbb{R}$ is denoted by $\mathcal{G}_{A,c,\sigma}$.

Hereafter, we say that a function $f : \mathbb{R} \to \mathbb{R}$ is *smooth* if $f \in \mathcal{G}_{A,c,\sigma}$ with some $A, c \in \mathbb{R}_+$ and $\sigma \in \mathbb{R}$.

Now, let us make some comments on the aforementioned setup. First, although we assumed that the domain of the second variable (the parameter) of F is $\mathbb{R}$, it might be some subset U of $\mathbb{R}$ in an actual problem. For example, when we consider financial derivative pricing and regard the parameter x as the initial underlying asset price, x is often limited to $\mathbb{R}_+$. Besides, unlike [17], Definition 1 refers to functions on $\mathbb{R}$ but not those on any subsets. These are just because we want to avoid cumbersomeness concerning to boundaries, which is mentioned in Section II B. We expect that we can extend the following discussion to the case of more general domains with no modification in essence. We also mention that we can often set the domain to $\mathbb{R}$ by variable transformation such as $\log x$, which maps $x \in \mathbb{R}_+$ into $\mathbb{R}$.

Second, note that the condition (23) is slightly different from that in [17]. That is, we introduced a constant A, which does not appear in [17]. This A bounds the value of the function f itself, and therefore informally represents the 'typical scale' of f. On the other hand, c^{-1} roughly represents the 'scale of variation of f'. In other words, we can informally say that $f(x)$ changes by roughly A when the variable x changes by c^{-1}. This view is helpful especially when the function f and the variable x have different dimensions, e.g., in financial derivative pricing, f as the price is written in the unit such as \$ and €, whereas the parameters such as the volatility have different units.

Third, Definition 1 is different from [17] also in that it does not cover differentiation of multivariate functions by multiple variables, whereas [17] does. This is because, in this paper, we focus on differentiation by a single variable for simplicity. Note that, even for a multivariate function, when we differentiate it with respect to one of its variables, we can treat it as univariate by fixing remaining variables. We expect that it is straightforward to extend the following discussion to cross partial derivatives using the difference formulas for them.

Finally, let us comment on smoothness of F. One might think that it is more natural to put smoothness conditions on $F(s,x)$ as a function of x than on V. However, in practice, it is possible that F is not smooth but V is. For example, in the aforementioned financial derivative pricing example, f_{pay} is often nonsmooth or even discontinuous, e.g. $f_{\text{pay}}(P) = (P -$

$K)^+$ for a call option and

$$f_{\text{pay}}(P) = \begin{cases} 1 & ; \text{ if } P \geq K \\ 0 & ; \text{ if } P < K \end{cases}$$

for a digital option, where K is some real number. In such a case, $F(s, x)$ is also nonsmooth or discontinuous with respect to x. Nevertheless, V in (16) is smooth with respect to P_0, σ and r, because of its definition as the integral of $\phi_{\text{SN}}(s)F(s, x)$ with respect to s. We may concern that, even though the original V is such an integral, we now consider V in (14), which is a finite sum arising from the discrete approximation, and therefore V is nonsmooth if so is F. On the other hand, as explained in Section II C, we now consider that the difference between V in (14) and the original V is negligible, and therefore so is the difference between the numerical differentiation values of these. Hence, in the following error analysis of the central difference formula for V in (14), we presume that it has the smoothness property as the original V.

B. Lemmas

Here, we present some lemmas for later use. Lemmas 2, 1, and 3 are proven in Appendices A, B and C, respectively.

Lemma 1. *Let m and n be positive integers satisfying $m \leq 2n$. Let $f \in C^{2n+1}(\mathbb{R})$ be a function satisfying the following: there exists $M \in \mathbb{R}$ such that*

$$\left|f^{(2n+1)}(x)\right| \leq M \tag{24}$$

holds for any $x \in \mathbb{R}$. Then, for any $h \in \mathbb{R}_+$ and any $x \in \mathbb{R}$, $R_f(x, n, m, h)$ given as (6) satisfies

$$|R_f(x, n, m, h)| \leq Mm\left(\frac{em}{2}\right)^{2n}. \tag{25}$$

Lemma 2. *For any positive integers m and n satisfying $m \leq 2n$, define*

$$D_n^{(m)} := \sum_{j=-n}^{n} \left|d_{n,j}^{(m)}\right| \tag{26}$$

with $d_{n,j}^{(m)}$ in (6). Then,

$$D_n^{(m)} \leq 2m\left[2\left(1 + \log n\right)\right]^m \tag{27}$$

holds.

Lemma 3. *Let $f : \mathbb{R} \to \mathbb{R}$ be in $\mathcal{G}_{A,c,\sigma}$ for some $A, c \in \mathbb{R}_+$ and $\sigma \in \mathbb{R}$. Then, for any $x \in \mathbb{R}$, $\epsilon \in \mathbb{R}_+$, $m \in \mathbb{N}$, $n \in \mathbb{N}$, and $h \in \mathbb{R}_+$ satisfying*

$$\epsilon' \leq \begin{cases} 2^{m\sigma^+ - \left(\frac{m\sigma^+}{\log 2}\right)^2} & ; \text{ if } m\sigma^+ \geq \log 2 \\ 2^{m\sigma^+ - 1} & ; \text{ otherwise} \end{cases}, \tag{28}$$

$$h \leq h_{\text{th}} := \frac{1}{ecm(2n+1)^{\sigma^+}}, \tag{29}$$

and

$$n \geq n_{\text{th}} := \left\lceil \frac{1}{2}\left[\log_2\left(\frac{2^{m\sigma^+}}{\epsilon'}\right) + \log_2\left(\log_2\left(\frac{2^{m\sigma^+}}{\epsilon'}\right)\right) - \frac{1}{2}\right]\right\rceil, \tag{30}$$

where

$$\epsilon' := \frac{e}{2(ecm)^m} \cdot \frac{\epsilon}{A}, \tag{31}$$

the following holds:

$$\left|f^{(m)}(x) - \mathcal{D}_{n,m,h}[f](x)\right| \leq \epsilon. \tag{32}$$

C. The naive iteration method

Now, let us consider quantum methods to compute $V^{(m)}(x)$ by numerical differentiation. Naively thinking, we conceive the following method, which we hereafter call the *naive iteration method*. That is, taking appropriate $n \in \mathbb{N}_{\geq \frac{m}{2}}$ and $h \in \mathbb{R}_+$ and supposing that we are given an oracle O_F in (19) (in reality, $O_{F,\epsilon}$ in (20)), we construct an oracle which computes $\mathcal{D}_{n,m,h}[F(s, \cdot)](x)$ in (18) by iteratively calling O_F for $x + (-n)h, x + (-n+1)h, ..., x + nh$. Then, using QAE with this oracle, we can estimate $\mathcal{D}_{n,m,h}[V](x)$ in (17) as an approximation of $V^{(m)}(x)$.

However, there is the following subtlety in this way. As mentioned above, it is common that F is nonsmooth, and therefore $\mathcal{D}_{n,m,h}[F(s, \cdot)](x)$ is not bounded for small h. For example, if $F(s, x)$ has a discontinuity with respect to x at some $(s', x') \in \mathcal{S} \times \mathbb{R}$, $\mathcal{D}_{1,1,h}[F(s', \cdot)](x) = \frac{1}{2h}\left(F(s', x' + h) - F(s', x' - h)\right)$ diverges when $h \to 0$. On the other hand, as explained in Section II C, in QMCI, we need to take some upper bound on the absolute values of the integrand and normalize it with the bound, in order to encode an integrand value into an amplitude of an ancilla qubit. This provides the difference between the cases where F is smooth and nonsmooth.

1. *The case of a smooth integrand*

First, we consider the smooth integrand case. In this case, $\mathcal{D}_{n,m,h}[F(s, \cdot)](x)$ is bounded, and therefore we can normalize it, and then estimate its expected value by QAE. We give the formal statement on the quantum algorithm in this case as follows, presenting the concrete calculation procedure in the proof.

Theorem 2. *Let $\mathcal{S}$ be some finite set such that each $s \in \mathcal{S}$ is associated with $p_s \in [0, 1]$ satisfying $\sum_{s \in \mathcal{S}} p_s = 1$. Let F be a real-valued function on $\mathcal{S} \times \mathbb{R}$ satisfying the following conditions:*

(i) there exists $B \in \mathbb{R}$ such that $|F(s, x)| \leq B$ holds for every $(s, x) \in \mathcal{S} \times \mathbb{R}$,

(ii) there exist $A, c \in \mathbb{R}_+$ and $\sigma \in \mathbb{R}$ such that, for every $s \in \mathcal{S}$, $F(s,x)$ as a function of x is in $\mathcal{G}_{A,c,\sigma}$.

Suppose that we have an access to an oracle O_S, which performs the operation (9). Suppose that, for any given $\epsilon \in \mathbb{R}_+$, we have an access to an oracle $O_{F,\epsilon}$, which performs the operation (20) for every $(s,x) \in \mathcal{S} \times \mathbb{R}$ with some function $F_\epsilon : \mathcal{S} \times \mathbb{R} \to \mathbb{R}$ satisfying (21) and uses qubits at most (22) with $a \in \mathbb{R}_+$. Suppose that we are given $x \in \mathbb{R}$, $m \in \mathbb{N}$ and $\epsilon \in \mathbb{R}_+$. Then, for any $n \in \mathbb{N}_{\geq \frac{m}{2}}$ and $h \in \mathbb{R}_+$ satisfying

$$Ac^{2n+1}((2n+1)!)^\sigma m \left(\frac{em}{2}\right)^{2n} h^{2n-m+1} \leq \epsilon, \tag{33}$$

there is a quantum algorithm $\mathcal{A}_1(m,\epsilon;n,h)$, which outputs 3ϵ-approximation of $V^{(m)}(x)$ with probability at least 0.99, making

$$O\left(\frac{Ac^m (m!)^\sigma}{\epsilon}\right) \tag{34}$$

calls to O_S and

$$O\left(\frac{Ac^m (m!)^\sigma n}{\epsilon}\right) \tag{35}$$

calls to $O_{F,\tilde{\epsilon}}$ and using qubits at most

$$O\left(\log^a\left(\frac{m(2(1+\log n))^m}{h^m \epsilon}\right)\right) \tag{36}$$

for $O_{F,\tilde{\epsilon}}$. Here,

$$\tilde{\epsilon} := \frac{h^m \epsilon}{D_n^{(m)}}. \tag{37}$$

In particular, $\mathcal{A}_1\left(m,\epsilon;\left\lceil \frac{m}{2}\right\rceil, h_{\min}\right)$, where

$$h_{\min} := \begin{cases} \left(\dfrac{\epsilon}{Ac((m+2)!)^\sigma m}\left(\dfrac{2}{ecm}\right)^{m+1}\right)^{1/2} & ;\ \text{if } m \text{ is odd} \\ \dfrac{\epsilon}{Ac((m+1)!)^\sigma m}\left(\dfrac{2}{ecm}\right)^{m} & ;\ \text{if } m \text{ is even} \end{cases}, \tag{38}$$

makes

$$O\left(\frac{Ac^m (m!)^\sigma m}{\epsilon}\right) \tag{39}$$

calls to $O_{F,\tilde{\epsilon}}$ and uses

$$\begin{cases} O\left(\log^a\left(\dfrac{e^{\frac{m^2+m}{2}} m^{\frac{1}{2}m^2+m+1} A^{\frac{m}{2}} c^{\frac{1}{2}m^2+m}((m+2)!)^{\frac{m\sigma}{2}}\left[\left(1+\log\left(\frac{m+1}{2}\right)\right)\right]^m}{2^{\frac{m^2-m}{2}}\epsilon^{\frac{m}{2}+1}}\right)\right) & ;\ \text{if } m \text{ is odd} \\ O\left(\log^a\left(\dfrac{e^{m^2} m^{m^2+m+1} A^m c^{m^2+m}((m+1)!)^{m\sigma}\left[\left(1+\log\left(\frac{m}{2}\right)\right)\right]^m}{2^{m^2}\epsilon^{m+1}}\right)\right) & ;\ \text{if } m \text{ is even} \end{cases} \tag{40}$$

qubits for $O_{F,\tilde{\epsilon}}$, and $\mathcal{A}_1(m,\epsilon;n_{\text{th}},h_{\text{th}})$, where n_{th} and h_{th} are given as (30) and (29) respectively, makes

$$O\left(\frac{Ac^m (m!)^\sigma m}{\epsilon}\log_2\left(\frac{2^{m\sigma^+}}{\epsilon'}\right)\right) \tag{41}$$

calls to $O_{F,\tilde{\epsilon}}$ and uses

$$O\left(\log^a\left(\frac{m(2ecm)^m B}{\epsilon}\right)\right) \tag{42}$$

qubits for $O_{F,\tilde{\epsilon}}$, where ϵ' is given as (31).

Proof. We first present the algorithm, and then consider the accuracy and the complexity.

Algorithm

Consider a system consisting of five quantum registers $R_1, ..., R_5$ and some ancillary registers as necessary. R_1 to R_4 have sufficient numbers of qubits, whereas R_5 has a single qubit. We can perform the following operation to the system initialized to $|0\rangle|0\rangle|0\rangle|0\rangle|0\rangle$:

Procedure 1

1: Using O_S, generate $\sum_{s\in\mathcal{S}} \sqrt{p_s}\,|s\rangle$ on R_1.
2: **for each** $i \in \mathcal{J}_{n,m}^{\neq 0} := \{j \in [-n:n] \mid d_{n,j}^{(m)} \neq 0\}$ **do**
3: Set R_2 to $|x+ih\rangle$.
4: By $O_{F,\tilde{\epsilon}}$, compute $F_{\tilde{\epsilon}}(s, x+ih)$ onto R_3, using the values on R_1 and R_2 as inputs.
5: By a multiplier circuit (e.g. [26, 32, 35]), add the product of $d_{n,i}^{(m)}$ and the value on R_3 to R_4.
6: By $O_{F,\tilde{\epsilon}}^{-1}$, uncompute R_3 to $|0\rangle$.
7: **end for**
8: By arithmetic circuits and a controlled rotation gate, transform the state on R_5 to

$$\sqrt{\frac{1}{2} + \frac{X}{2h^m(Ac^m(m!)^\sigma + 2\epsilon)}}\,|1\rangle + \sqrt{\frac{1}{2} - \frac{X}{2h^m(Ac^m(m!)^\sigma + 2\epsilon)}}\,|0\rangle. \tag{43}$$

using the value on R_4 as X.

We denote the oracle that corresponds to this operation as Q_1. In this operation, the quantum state is transformed as follows:

$$
\begin{aligned}
& |0\rangle|0\rangle|0\rangle|0\rangle|0\rangle \\
\xrightarrow{1} \quad & \sum_{s\in\mathcal{S}} \sqrt{p_s}\,|s\rangle|0\rangle|0\rangle|0\rangle|0\rangle \\
\xrightarrow{3 \text{ for } i=-n} \quad & \sum_{s\in\mathcal{S}} \sqrt{p_s}\,|s\rangle|x+(-n)h\rangle|0\rangle|0\rangle|0\rangle \\
\xrightarrow{4 \text{ for } i=-n} \quad & \sum_{s\in\mathcal{S}} \sqrt{p_s}\,|s\rangle|x+(-n)h\rangle|F_{\tilde{\epsilon}}(s,x+(-n)h)\rangle|0\rangle|0\rangle \\
\xrightarrow{5 \text{ for } i=-n} \quad & \sum_{s\in\mathcal{S}} \sqrt{p_s}\,|s\rangle|x+(-n)h\rangle|F_{\tilde{\epsilon}}(s,x+(-n)h)\rangle|d^{(m)}_{n,-n}F_{\tilde{\epsilon}}(s,x+(-n)h)\rangle|0\rangle \\
\xrightarrow{6 \text{ for } i=-n} \quad & \sum_{s\in\mathcal{S}} \sqrt{p_s}\,|s\rangle|x+(-n)h\rangle|0\rangle|d^{(m)}_{n,-n}F_{\tilde{\epsilon}}(s,x+(-n)h)\rangle|0\rangle \\
\xrightarrow{3 \text{ for } i=-n+1} \quad & \sum_{s\in\mathcal{S}} \sqrt{p_s}\,|s\rangle|x+(-n+1)h\rangle|0\rangle|d^{(m)}_{n,-n}F_{\tilde{\epsilon}}(s,x+(-n)h)\rangle|0\rangle \\
\xrightarrow{4 \text{ for } i=-n+1} \quad & \sum_{s\in\mathcal{S}} \sqrt{p_s}\,|s\rangle|x+(-n+1)h\rangle|F_{\tilde{\epsilon}}(s,x+(-n+1)h)\rangle|d^{(m)}_{n,-n}F_{\tilde{\epsilon}}(s,x+(-n)h)\rangle|0\rangle \\
\xrightarrow{5 \text{ for } i=-n+1} \quad & \sum_{s\in\mathcal{S}} \sqrt{p_s}\,|s\rangle|x+(-n+1)h\rangle|F_{\tilde{\epsilon}}(s,x+(-n+1)h)\rangle\left|\sum_{j=-n}^{-n+1} d^{(m)}_{n,j}F_{\tilde{\epsilon}}(s,x+jh)\right\rangle|0\rangle \\
\xrightarrow{6 \text{ for } i=-n+1} \quad & \sum_{s\in\mathcal{S}} \sqrt{p_s}\,|s\rangle|x+(-n+1)h\rangle|0\rangle\left|\sum_{j=-n}^{-n+1} d^{(m)}_{n,j}F_{\tilde{\epsilon}}(s,x+jh)\right\rangle|0\rangle \\
\rightarrow \quad & \cdots \\
\xrightarrow{6 \text{ for } i=n} \quad & \sum_{s\in\mathcal{S}} \sqrt{p_s}\,|s\rangle|x+nh\rangle|0\rangle\left|\sum_{j=-n}^{n} d^{(m)}_{n,j}F_{\tilde{\epsilon}}(s,x+jh)\right\rangle|0\rangle \\
\xrightarrow{8} \quad & \sum_{s\in\mathcal{S}} \sqrt{p_s}\,|s\rangle|x+nh\rangle|0\rangle\left|\sum_{j=-n}^{n} d^{(m)}_{n,j}F_{\tilde{\epsilon}}(s,x+jh)\right\rangle \\
& \otimes\left(\sqrt{\frac{1}{2}+\frac{1}{2h^m(Ac^m(m!)^\sigma+2\epsilon)}\sum_{j=-n}^{n} d^{(m)}_{n,j}F_{\tilde{\epsilon}}(s,x+jh)}\,|1\rangle+\sqrt{\frac{1}{2}-\frac{1}{2h^m(Ac^m(m!)^\sigma+2\epsilon)}\sum_{j=-n}^{n} d^{(m)}_{n,j}F_{\tilde{\epsilon}}(s,x+jh)}\,|0\rangle\right) \\
=: \quad & |\Psi_1\rangle\,,
\end{aligned}
\tag{44}
$$

Note that the insides of the square roots in the last line in (44) are in [0, 1], since

$$
\begin{aligned}
& \left|\frac{1}{h^m(Ac^m(m!)^\sigma+2\epsilon)}\sum_{j=-n}^{n} d^{(m)}_{n,j}F_{\tilde{\epsilon}}(s,x+jh)\right| \\
\le & \left|\frac{1}{h^m(Ac^m(m!)^\sigma+2\epsilon)}\sum_{j=-n}^{n} d^{(m)}_{n,j}F(s,x+jh)\right| + \\
& \left|\frac{1}{h^m(Ac^m(m!)^\sigma+2\epsilon)}\sum_{j=-n}^{n} d^{(m)}_{n,j}\left(F(s,x+jh)-F_{\tilde{\epsilon}}(s,x+jh)\right)\right| \\
\le & \frac{\left|\mathcal{D}_{n,m,h}[F(s,\cdot)](x)\right|}{Ac^m(m!)^\sigma+2\epsilon} + \\
& \frac{1}{h^m(Ac^m(m!)^\sigma+2\epsilon)}\sum_{j=-n}^{n}\left|d^{(m)}_{n,j}\right|\cdot|F(s,x+jh)-F_{\tilde{\epsilon}}(s,x+jh)| \\
\le & \frac{Ac^m(m!)^\sigma+\epsilon}{Ac^m(m!)^\sigma+2\epsilon}+\frac{1}{h^m(Ac^m(m!)^\sigma+2\epsilon)}D^{(m)}_n\tilde{\epsilon} \\
= & 1.
\end{aligned}
\tag{45}
$$

Here, at the third inequality, we used

$$
\begin{aligned}
& \left|\mathcal{D}_{n,m,h}[F(s,\cdot)](x)\right| \\
\le & \left|\frac{\partial^m F(s,x)}{\partial x^m}\right| + \left|\mathcal{D}_{n,m,h}[F(s,\cdot)](x)-\frac{\partial^m F(s,x)}{\partial x^m}\right| \\
\le & Ac^m(m!)^\sigma+\epsilon,
\end{aligned}
$$

which follows from $F(s,\cdot)\in\mathcal{G}_{A,c,\sigma}$, Lemma 1 and (33). The probability that we obtain 1 on the last qubit in measuring $|\Psi_1\rangle$ is

$$
P := \frac{1}{2}+\frac{1}{2h^m(Ac^m(m!)^\sigma+2\epsilon)}\sum_{j=-n}^{n} d^{(m)}_{n,j}\sum_{s\in\mathcal{S}} p_s F_{\tilde{\epsilon}}(s,x+jh). \tag{46}
$$

Defining

$$Y := (Ac^m(m!)^\sigma + 2\epsilon)(2P - 1), \tag{47}$$

we see that

$$\begin{aligned}
&\left|\mathcal{D}_{n,m,h}[V](x) - Y\right| \\
&\leq \sum_{j=-n}^{n} \sum_{s\in\mathcal{S}} \frac{1}{h^m} \left|d_{n,j}^{(m)}\right| p_s \left|F(s, x + jh) - F_{\tilde{\epsilon}}(s, x + jh)\right| \\
&\leq \frac{1}{h^m} D_n^{(m)} \tilde{\epsilon} \\
&= \epsilon.
\end{aligned} \tag{48}$$

Therefore, we obtain an estimate of $\mathcal{D}_{n,m,h}[V](x)$ as follows: obtain an estimate $\tilde{P}$ of P by QAE, in which Q_1 is iteratively called, and then output

$$\tilde{Y} := (Ac^m(m!)^\sigma + 2\epsilon)(2\tilde{P} - 1). \tag{49}$$

Accuracy and complexity

$$\begin{aligned}
&|V^{(m)}(x) - \mathcal{D}_{n,m,h}[V](x)| \\
&\leq |R_V(x, n, m, h)| h^{2n-m+1} \\
&\leq Ac^{2n+1}((2n + 1)!)^\sigma m \left(\frac{em}{2}\right)^{2n} h^{2n-m+1} \\
&\leq \epsilon
\end{aligned} \tag{50}$$

holds. Here, the first inequality holds because of Theorem 1. At the second inequality, we use Lemma 1 with $M = Ac^{2n+1}((2n + 1)!)^\sigma$, since $V \in \mathcal{G}_{A,c,\sigma}$ as easily seen from $F(s, \cdot) \in \mathcal{G}_{A,c,\sigma}$ for every $s \in \mathcal{S}$. The last inequality is (33). Using (50) and (48), we see that, if we have $\tilde{Y}$ such that

$$|\tilde{Y} - Y| \leq \epsilon, \tag{51}$$

the following holds:

$$\begin{aligned}
&|V^{(m)}(x) - \tilde{Y}| \\
&\leq |V^{(m)}(x) - \mathcal{D}_{n,m,h}[V](x)| + |\mathcal{D}_{n,m,h}[V](x) - Y| + |Y - \tilde{Y}| \\
&\leq 3\epsilon,
\end{aligned} \tag{52}$$

which means that $\tilde{Y}$ is an 3ϵ-approximation of $V^{(m)}(x)$.

Then, let us estimate the query complexity to obtain $\tilde{P}$ such that (51) holds by QAE. Because of the definitions (47) and (49), it is sufficient to obtain $\tilde{P}$ such that

$$|\tilde{P} - P| \leq \frac{\epsilon}{2(Ac^m(m!)^\sigma + 2\epsilon)} \tag{53}$$

by QAE. For this, QAE with N_{Q_1} calls to Q_1, where N_{Q_1} is at most (34), is sufficient. Since Q_1 uses O_S once and $O_{F,\tilde{\epsilon}}$ at most $2n + 1$ times, we evaluate the numbers of queries to them as (34) and (35). We also have (36), combining the assumption that $O_{F,\tilde{\epsilon}}$ uses $O\left(\log^a\left(\frac{1}{\tilde{\epsilon}}\right)\right)$ qubits with (37) and Lemma 2.

For $n = \left\lceil \frac{m}{2} \right\rceil$ and $h = h_{\min}$, which we can check satisfy (33) by simple algebra, we just plug these values into (35) and (36), and then obtain (39) and (2), respectively (note that the number of queries to O_S does not depend on n and h).

Also for $n = n_{\text{th}}$ and $h = h_{\text{th}}$, for which (33) holds as shown in the proof of Lemma 3 (see (C15)), just plugging these values into (35) and (36) with some algebra leads to (41) and (42), respectively.

□

(34) indicates that, asymptotically, the upper bound on the number of queries to O_S scales ϵ as $O\left(\frac{1}{\epsilon}\right)$ and independent from n and h. Besides, from (35), we see that the query number bound for $O_{F,\tilde{\epsilon}}$ depends on not h but n linearly, and scales as $O\left(\frac{1}{\epsilon}\right)$ if n does not depend on ϵ. Hence, setting n to the minimum value $\left\lceil \frac{m}{2} \right\rceil$ and h to (38) is best in the aspect of this bound. However, the qubit number for $O_{F,\tilde{\epsilon}}$ becomes large in this setting, depending on ϵ as $O\left(\log^a\left(\frac{1}{\epsilon^{\frac{m}{2}+1}}\right)\right)$ or $O\left(\log^a\left(\frac{1}{\epsilon^{m+1}}\right)\right)$. On the other hand, setting $n = n_{\text{th}}$ and $h = h_{\text{th}}$ leads to less qubit number scaling as $O\left(\log^a\left(\frac{1}{\epsilon}\right)\right)$, adding a $O\left(\log\left(\frac{1}{\epsilon}\right)\right)$ factor to the query number bound for $O_{F,\tilde{\epsilon}}$.

2. *The case of a nonsmooth integrand*

Next, we consider the nonsmooth integrand case. Now, $\mathcal{D}_{n,m,h}[F(s, \cdot)](x)$ can be unbounded when $h \to 0$, and therefore, in the algorithm we propose, we estimate the expectation value of not $\mathcal{D}_{n,m,h}[F(s, \cdot)](x)$ but $\sum_{j=-n}^{n} d_{n,j}^{(m)} F(s, x + jh)$, omitting the factor $1/h^m$, and then divide the estimate by h^m to obtain $\mathcal{D}_{n,m,h}[V](x)$. We present the formal statement on this method as follows.

Theorem 3. *Let $\mathcal{S}, x, m$ and ϵ be as described in Theorem 2. Let $F : \mathcal{S} \times \mathbb{R} \to \mathbb{R}$ be a function satisfying the condition (i) in Theorem 2 and the following*

(ii)' there exist $A, c \in \mathbb{R}_+$ and $\sigma \in \mathbb{R}$ such that V defined as (14) is in $\mathcal{G}_{A,c,\sigma}$.

Suppose that we are given accesses to the oracles O_S and $O_{F,\epsilon}$ for any $\epsilon \in \mathbb{R}_+$, which are described in Theorem 2. Then, for any $n \in \mathbb{N}_{\geq \frac{m}{2}}$ and $h \in \mathbb{R}_+$ satisfying (33), there is a quantum algorithm $\mathcal{A}_2(m, \epsilon; n, h)$, which outputs 3ϵ-approximation of $V^{(m)}(x)$ with probability at least 0.99, making

$$O\left(\frac{m\left[2\left(1 + \log n\right)\right]^m B}{h^m \epsilon}\right) \tag{54}$$

calls to O_S and

$$O\left(\frac{mn\left[2\left(1 + \log n\right)\right]^m B}{h^m \epsilon}\right) \tag{55}$$

calls to $O_{F,\tilde{\epsilon}}$ and using qubits at most (36) for $O_{F,\tilde{\epsilon}}$. Here, $\tilde{\epsilon}$ is given as (37). In particular, $\mathcal{A}_2\left(m, \epsilon; \left\lceil \frac{m}{2} \right\rceil, h_{\min}\right)$, where $h_{\min}$

is given as (38), makes

$$\begin{cases} O\left(\frac{e^{\frac{1}{2}m(m+1)}m^{\frac{1}{2}m^2+m+1}A^{\frac{m}{2}}c^{\frac{1}{2}m(m+2)}((m+2)!)^{m\sigma/2}\left[2\left(1+\log\left(\frac{m+1}{2}\right)\right)\right]^m B}{2^{\frac{1}{2}m(m+1)}\epsilon^{\frac{m}{2}+1}}\right) & ;\text{ if } m \text{ is odd} \\ O\left(\frac{e^{m^2}m^{m^2+m+1}A^m c^{m^2+m}((m+1)!)^{m\sigma}\left[2\left(1+\log\left(\frac{m}{2}\right)\right)\right]^m B}{2^{m^2}\epsilon^{m+1}}\right) & ;\text{ if } m \text{ is even} \end{cases} \tag{56}$$

calls to O_S and

$$\begin{cases} O\left(\frac{e^{\frac{1}{2}m(m+1)}m^{\frac{1}{2}m^2+m+2}A^{\frac{m}{2}}c^{\frac{1}{2}m(m+2)}((m+2)!)^{m\sigma/2}\left[2\left(1+\log\left(\frac{m+1}{2}\right)\right)\right]^m B}{2^{\frac{1}{2}m(m+1)}\epsilon^{\frac{m}{2}+1}}\right) & ;\text{ if } m \text{ is odd} \\ O\left(\frac{e^{m^2}m^{m^2+m+2}A^m c^{m^2+m}((m+1)!)^{m\sigma}\left[2\left(1+\log\left(\frac{m}{2}\right)\right)\right]^m B}{2^{m^2}\epsilon^{m+1}}\right) & ;\text{ if } m \text{ is even} \end{cases} \tag{57}$$

calls to $O_{F,\tilde{\epsilon}}$ and uses qubits at most (2) for $O_{F,\tilde{\epsilon}}$, and $\mathcal{A}_2(m,\epsilon;n_{\rm th},h_{\rm th})$, where $n_{\rm th}$ and $h_{\rm th}$ are given as (30) and (29) respectively, makes

$$O\left(\frac{m(2ecm)^m B}{\epsilon}\log_2^{m\sigma^+}\left(\frac{2^{m\sigma^+}}{\epsilon'}\right)\log^m\left(\log_2\left(\frac{2^{m\sigma^+}}{\epsilon'}\right)\right)\right) \tag{58}$$

calls to O_S and

$$O\left(\frac{m(2ecm)^m B}{\epsilon}\log_2^{m\sigma^++1}\left(\frac{2^{m\sigma^+}}{\epsilon'}\right)\log^m\left(\log_2\left(\frac{2^{m\sigma^+}}{\epsilon'}\right)\right)\right) \tag{59}$$

calls to $O_{F,\tilde{\epsilon}}$ and uses qubits at most (42) for $O_{F,\tilde{\epsilon}}$, where ϵ' is given as (31).

Proof. We first present the algorithm, and then consider the accuracy and the complexity.

Algorithm

Consider a system consisting of same quantum registers $R_1,...,R_5$ as Theorem 2 and some ancillary registers as necessary. We can perform the operation same as Procedure 1 except the replacement of (60) with

$$\sqrt{\frac{1}{2}+\frac{X}{2(D_n^{(m)}B+\tilde{\epsilon})}}\,|1\rangle+\sqrt{\frac{1}{2}-\frac{X}{2(D_n^{(m)}B+\tilde{\epsilon})}}\,|0\rangle\,, \tag{60}$$

We denote the oracle that corresponds to this operation as Q_2. By this operation, the quantum state is transformed as follows:

$$\begin{aligned} &|0\rangle|0\rangle|0\rangle|0\rangle|0\rangle \\ &\to \sum_{s\in\mathcal{S}}\sqrt{p_s}\,|s\rangle|x+nh\rangle|0\rangle\left|\sum_{j=-n}^{n}d_{n,j}^{(m)}F_{\tilde{\epsilon}}(s,x+jh)\right\rangle \\ &\quad\otimes\left(\sqrt{\frac{1}{2}+\frac{1}{2D_n^{(m)}(B+\tilde{\epsilon})}\sum_{j=-n}^{n}d_{n,j}^{(m)}F_{\tilde{\epsilon}}(s,x+jh)}\,|1\rangle+\sqrt{\frac{1}{2}-\frac{1}{2D_n^{(m)}(B+\tilde{\epsilon})}\sum_{j=-n}^{n}d_{n,j}^{(m)}F_{\tilde{\epsilon}}(s,x+jh)}\,|0\rangle\right)=:|\Psi_2\rangle\,. \end{aligned} \tag{61}$$

Note that the insides of the square roots in the last line in (61) are in [0, 1], since

$$\begin{aligned} &\left|\frac{1}{D_n^{(m)}(B+\tilde{\epsilon})}\sum_{j=-n}^{n}d_{n,j}^{(m)}F_{\tilde{\epsilon}}(s,x+jh)\right| \\ &\le\left|\frac{1}{D_n^{(m)}(B+\tilde{\epsilon})}\sum_{j=-n}^{n}d_{n,j}^{(m)}F(s,x+jh)\right| \\ &\quad+\left|\frac{1}{D_n^{(m)}(B+\tilde{\epsilon})}\sum_{j=-n}^{n}d_{n,j}^{(m)}\left(F(s,x+jh)-F_{\tilde{\epsilon}}(s,x+jh)\right)\right| \\ &\le\frac{1}{D_n^{(m)}(B+\tilde{\epsilon})}\sum_{j=-n}^{n}\left|d_{n,j}^{(m)}\right|\cdot|F(s,x+jh)| \\ &\quad+\frac{1}{D_n^{(m)}(B+\tilde{\epsilon})}\sum_{j=-n}^{n}\left|d_{n,j}^{(m)}\right|\cdot|F(s,x+jh)-F_{\tilde{\epsilon}}(s,x+jh)| \\ &\le\frac{1}{D_n^{(m)}(B+\tilde{\epsilon})}D^{(m)}B+\frac{1}{D_n^{(m)}(B+\tilde{\epsilon})}D^{(m)}\tilde{\epsilon} \\ &\le 1. \end{aligned} \tag{62}$$

The probability that we obtain 1 on the last qubit in measuring $|\Psi_2\rangle$ is

$$P:=\frac{1}{2}+\frac{1}{2D_n^{(m)}(B+\tilde{\epsilon})}\sum_{j=-n}^{n}d_{n,j}^{(m)}\sum_{s\in\mathcal{S}}p_sF_{\tilde{\epsilon}}(s,x+jh). \tag{63}$$

Defining

$$Y:=\frac{D_n^{(m)}(B+\tilde{\epsilon})}{h^m}(2P-1), \tag{64}$$

we see that $\left|\mathcal{D}_{n,m,h}[V](x) - Y\right| \leq \epsilon$ similarly to (48). Therefore, we obtain an estimate of $\mathcal{D}_{n,m,h}[V](x)$ as follows: obtain an estimate $\tilde{P}$ of P by QAE, in which Q_2 is iteratively called, and then output

$$\tilde{Y} := \frac{D_n^{(m)}(B+\tilde{\epsilon})}{h^m}(2\tilde{P}-1). \tag{65}$$

Accuracy and complexity

As shown in the proof of Theorem 2, if we have $\tilde{Y}$ such that $|\tilde{Y} - Y| \leq \epsilon$, $\tilde{Y}$ is an 3ϵ-approximation of $V^{(m)}(x)$. Then, let us estimate the query complexity to obtain $\tilde{P}$ that makes this hold by QAE. Because of the definitions (64) and (65), it is sufficient to obtain $\tilde{P}$ such that

$$|\tilde{P} - P| \leq \frac{h^m}{2D_n^{(m)}(B+\tilde{\epsilon})}\epsilon \tag{66}$$

by QAE. For this, QAE with N_{Q_2} calls to Q_2, where

$$N_{Q_2} = O\left(\frac{D_n^{(m)}(B+\tilde{\epsilon})}{h^m\epsilon}\right), \tag{67}$$

is sufficient. Using Lemma 2 and (33) with simple algebra, we see that (67) is evaluated as (54). Since Q_2 uses O_S once and $O_{F,\tilde{\epsilon}}$ at most $2n+1$ times, we have (54) and (55). We also prove the claim on the qubit number for $O_{F,\tilde{\epsilon}}$ similarly to Theorem 2.

Remaining claims for the specific settings on n and h are proven by simply plugging their values into the expressions of the query numbers and the qubit number.

□

Since we multiply the result of QAE by $\frac{D_n^{(m)}(B+\tilde{\epsilon})}{h^m}$, this factor is included also in the upper bounds (54) and (55) on the query numbers in this method. In the case of $n = \left\lceil \frac{m}{2} \right\rceil$, the minimum value of n, this increases the exponent of $\frac{1}{\epsilon}$ in the query number bounds, since we set h to a small value depending on ϵ as (38). Conversely, if we set n efficiently large as $n = n_{\mathrm{th}}$, we can set h to h_{th}, which depends on ϵ only logarithmically through n, and therefore the query number bounds (58) and (59) scale with ϵ as $O\left(\frac{1}{\epsilon}\right)$, expect the logarithmic factor. Besides, larger n and h lead to the smaller qubit number for $O_{F,\epsilon}$, similarly to the smooth integrand case. Hence, in the nonsmooth integrand case, setting $n = n_{\mathrm{th}}$ and $h = h_{\mathrm{th}}$ is better than setting to smaller numbers, in terms of both query complexity and qubit number.

D. The sum-in-QAE method

We now consider another quantum method for numerical differentiation of V, which we name the *sum-in-QAE* method. Note that (17) is a two-fold summation, which consists of the sum over the values s of the stochastic variable and the sum over j in the finite difference formula. Then, we can take the latter sum at the same time as the former in one QAE, unlike the iterative calls to $O_{F,\epsilon}$ for $x+(-n)h, ..., x+nh$ in the naive iteration method. We present the detail of this method in the following theorem.

Theorem 4. *Let $\mathcal{S}, x, m, \epsilon$ and F be as described in Theorem 3. Suppose that we are given accesses to the oracles O_S and $O_{F,\epsilon}$ for any $\epsilon \in \mathbb{R}_+$, which are described in Theorem 2 and 3. Besides, suppose that, for every $n \in N_{\frac{m}{2}}$, we have accesses to the oracles $O_{\mathrm{coef}}^{m,n}$, which performs the operation*

$$O_{\mathrm{coef}}^{m,n}|0\rangle = |\Psi_{\mathrm{coef}}^{n,m}\rangle := \frac{1}{\sqrt{D_n^{(m)}}}\sum_{j\in[-n:n]}\sqrt{\left|d_{n,j}^{(m)}\right|}|j\rangle, \tag{68}$$

with $d_{n,j}^{(m)}$ defined as (6), and $O_{\mathrm{sign}}^{m,n}$, which performs the operation

$$O_{\mathrm{sign}}^{m,n}|j\rangle|0\rangle = |j\rangle\left|\theta_j^{m,n}\right\rangle, \tag{69}$$

for every $j \in [-n:n]$ with

$$\theta_j^{m,n} := \begin{cases} 1 & ;\text{ if } d_{n,j}^{(m)} \geq 0 \\ 0 & ;\text{ otherwise} \end{cases} \tag{70}$$

Then, for any $n \in \mathbb{N}_{\geq \frac{m}{2}}$ and $h \in \mathbb{R}_+$ satisfying (33), there is a quantum algorithm $\mathcal{A}_3(m,\epsilon;n,h)$, which outputs 3ϵ-approximation of $V^{(m)}(x)$ with probability at least 0.99, making calls to O_S, $O_{F,\tilde{\epsilon}}$, $O_{\mathrm{coef}}^{m,n}$ and $O_{\mathrm{sign}}^{m,n}$ the number of times shown in (54), and using qubits at most (36) for $O_{F,\tilde{\epsilon}}$, with $\tilde{\epsilon}$ given as (37). In particular, $\mathcal{A}_3(m,\epsilon;n_{\mathrm{th}},h_{\mathrm{th}})$, where n_{th} and h_{th} are given as (30) and (29) respectively, calls O_S, $O_{F,\tilde{\epsilon}}$, $O_{\mathrm{coef}}^{m,n}$ and $O_{\mathrm{sign}}^{m,n}$ the number of times shown in (58), and uses qubits at most (42) for $O_{F,\tilde{\epsilon}}$, where ϵ' is given as (31).

Proof. Consider a system consisting of six quantum registers $R_1, ..., R_6$ and some ancillary registers as necessary. R_3 and R_6 have a single qubit, and the others have sufficient numbers of qubits. We can perform the following operation to the system initialized to $|0\rangle|0\rangle|0\rangle|0\rangle|0\rangle|0\rangle$:

Procedure 2

1: Using $O_{\mathrm{coef}}^{m,n}$, generate the state (68) on R_1.
2: Set $x+jh$ on R_2, using the value on R_1 as j.
3: By $O_{\mathrm{sign}}^{m,n}$, set $\theta_j^{m,n}$ on R_3, using the value on R_1 as j.
4: Using O_S, generate $\sum_{s\in\mathcal{S}}\sqrt{p_s}|s\rangle$ on R_4.
5: By $O_{F,\tilde{\epsilon}}$, compute $F_{\tilde{\epsilon}}(s, x+jh)$ onto R_5, using the values on R_2 and R_4 as inputs.
6: By a circuit similar to that in step 8 in Procedure 1 and a NOT gate on R_6 activated only if the value on R_3 is 0, transform the state on R_6 to

$$\sqrt{\frac{1}{2}+\frac{X}{2(B+\tilde{\epsilon})}}|\theta\rangle + \sqrt{\frac{1}{2}-\frac{X}{2(B+\tilde{\epsilon})}}|1-\theta\rangle, \tag{71}$$

where θ and X are the values on R_3 and R_5, respectively.

We denote the oracle that corresponds to this operation by Q_3. In this operation, the quantum state is transformed as follows:

$$
\begin{aligned}
& |0\rangle|0\rangle|0\rangle|0\rangle|0\rangle|0\rangle \\
\xrightarrow{1} \quad & \frac{1}{\sqrt{D_n^{(m)}}} \sum_{j\in[-n:n]} \sqrt{\left|d_{n,j}^{(m)}\right|}\,|j\rangle|0\rangle|0\rangle|0\rangle|0\rangle|0\rangle \\
\xrightarrow{2 \text{ and } 3} \quad & \frac{1}{\sqrt{D_n^{(m)}}} \sum_{j\in[-n:n]} \sqrt{\left|d_{n,j}^{(m)}\right|}\,|j\rangle|x+jh\rangle|\theta_j^{m,n}\rangle|0\rangle|0\rangle|0\rangle \\
\xrightarrow{4} \quad & \frac{1}{\sqrt{D_n^{(m)}}} \sum_{j\in[-n:n]} \sum_{s\in\mathcal{S}} \sqrt{\left|d_{n,j}^{(m)}\right| p_s}\,|j\rangle|x+jh\rangle|\theta_j^{m,n}\rangle|s\rangle|0\rangle|0\rangle \\
\xrightarrow{5} \quad & \frac{1}{\sqrt{D_n^{(m)}}} \sum_{j\in[-n:n]} \sum_{s\in\mathcal{S}} \sqrt{\left|d_{n,j}^{(m)}\right| p_s}\,|j\rangle|x+jh\rangle|\theta_j^{m,n}\rangle|s\rangle|F_{\tilde{\epsilon}}(s,x+jh)\rangle|0\rangle \\
\xrightarrow{6} \quad & \frac{1}{\sqrt{D_n^{(m)}}} \sum_{j\in[-n:n]} \sum_{s\in\mathcal{S}} \sqrt{\left|d_{n,j}^{(m)}\right| p_s}\,|j\rangle|x+jh\rangle|\theta_j^{m,n}\rangle|s\rangle|F_{\tilde{\epsilon}}(s,x+jh)\rangle \\
& \otimes\left(\sqrt{\frac{1}{2}+\frac{F_{\tilde{\epsilon}}(s,x+jh)}{2(B+\tilde{\epsilon})}}\left|\theta_j^{m,n}\right\rangle + \sqrt{\frac{1}{2}-\frac{F_{\tilde{\epsilon}}(s,x+jh)}{2(B+\tilde{\epsilon})}}\left|1-\theta_j^{m,n}\right\rangle\right) =: |\Phi\rangle\,.
\end{aligned}
\tag{72}
$$

Note that the insides of the square roots in the last line in (72) are in $[0,1]$, since

$$|F_{\tilde{\epsilon}}(s,x+jh)| \le |F(s,x+jh)| + |F_{\tilde{\epsilon}}(s,x+jh) - F(s,x+jh)| \le B+\tilde{\epsilon}.$$

The probability that we obtain 1 on R_6 in measuring $|\Phi\rangle$ is equal to P in (63), and Y defined as (64) satisfies $\left|\mathcal{D}_{n,m,h}[V](x) - Y\right| \le \epsilon$ as seen in the proof of Theorem 3. Therefore, we get an estimate of $\mathcal{D}_{n,m,h}[V](x)$ as follows: obtain an estimate $\tilde{P}$ of P by QAE, in which Q_3 is iteratively called, and then output $\tilde{Y}$ in (65).

Accuracy and complexity

The sum-in-QAE method is same as the naive iteration method for nonsmooth F in that the quantity estimated by QAE is P in (63) and that the final output is $\tilde{Y}$ in (65) calculated with the estimate $\tilde{P}$ for P, although the iteratively called oracles Q_2 and Q_3 are different. Therefore, the number of calls to Q_3 in the sum-in-QAE method is equal to the number of calls to Q_2 in the naive iteration method for nonsmooth F, and is evaluated as (67), or (54). Since O_S, $O_{F,\tilde{\epsilon}}$, $O_{\text{coef}}^{m,n}$ and $O_{\text{sign}}^{m,n}$ are called once in Q_3, the numbers of calls to these oracles are also evaluated as (54). Since the same $\tilde{\epsilon}$ is used in the two methods, the qubit number for $O_{F,\tilde{\epsilon}}$ is also same, and given as (36).

The remaining claim on the specific case[2] that $n = n_{\text{th}}$ and $h = h_{\text{th}}$ is also proven similarly to the discussion in the proof of Theorem 3.

□

Compared with the number of calls to $O_{F,\tilde{\epsilon}}$ in the naive iteration method, which scales with n as $O(n)$ and $O(n \times \text{polylog}(n))$ in the cases of smooth and nonsmooth F, respectively, that in the sum-in-QAE method more mildly scales as $O(\text{polylog}(n))$. This is because the sum-in-QAE method takes the sum in central difference formula by QAE and the normalization factor $D_n^{(m)}$ in the QAE target state $|\Phi\rangle$ is $O(\text{polylog}(n))$.

Note that the sum-in-QAE method works for differentiation of expected values like V, or, more broadly, functions calculated by QAE, but not for general functions. That is, if a function f is defined as a summation like V and QAE is used for the sum, we can 'mix' the sum in the central difference formula into the sum in f, and simultaneously perform the two sums in one QAE. On the other hand, if f is calculated without the aid of QAE, we can calculate $\mathcal{D}_{n,m,h}[f](x)$ by naively iterating calculation and summation of $f(x+(-n)h), ..., f(x+nh)$ faster than the way that they are computed in quantum parallel and summed up by QAE. This is because the number of calls to f in the naive iteration is at most $2n+1$, which is $O\left(\text{polylog}\left(\frac{1}{\epsilon}\right)\right)$ even in the setting $n = n_{\text{th}}$ and $h = h_{\text{th}}$, but that in the QAE-based calculation is $O\left(\frac{1}{\epsilon}\right)$.

Let us comment also on implementation of $O_{\text{coef}}^{m,n}$ and $O_{\text{sign}}^{m,n}$. $O_{\text{coef}}^{m,n}$ is the circuit to load the $2n+1$ precalculated numbers $\left|d_{n,-n}^{(m)}\right|, ..., \left|d_{n,n}^{(m)}\right|$ as the quantum state $|\Psi_{\text{coef}}^{m,n}\rangle$, and implemented using $\Theta(n)$ elementary gates [41–47]. $O_{\text{sign}}^{m,n}$ is the circuit to return 1 or 0 for each $j \in [-n:n]$ in the predetermined way, and implemented by n multi-controlled NOT gates. Thus, for

[2] Although we considered the case that n is set to the minimum value $\left\lceil \frac{m}{2} \right\rceil$ in Theorem 3, we omit this case here, since this is less efficient than larger n in terms of both query complexity and qubit number, as we saw in the proof of Theorem 3.

n which is at most logarithmically large as $n_{\rm th}$, we regard these oracles as less time-consuming than $O_{F,\tilde{\epsilon}}$.

E. Comparison of the methods

Now, let us compare the presented methods in terms of the dependency of the number of queries to $O_{F,\tilde{\epsilon}}$ on error tolerance ϵ, and discuss which one is better in each case. Table I summarizes this, displaying the query numbers in various situations. We consider the cases of smooth and nonsmooth F. With respect to qubit capacity, we consider the situation where we can use as many qubits as we want and the opposite situation where the qubits available are limited and we want to save the qubit number. Then, we consider the two settings, $n = \left\lceil \frac{m}{2} \right\rceil, h = h_{\rm min}$ and $n = n_{\rm th}, h = h_{\rm th}$, and assume that the former is possible only in the large qubit capacity case. When F is nonsmooth, regardless of qubit capacity, the sum-in-QAE method is better than the naive iteration method by a factor $\log\left(\frac{1}{\epsilon}\right)$. On the other hand, when F is smooth and many qubits are available, the naive iteration method is better. The discussion in the case of smooth F and the small qubit capacity depends on comparison between factors $\log\left(\frac{1}{\epsilon}\right)$ and $\log^{m\sigma^+}\left(\frac{1}{\epsilon}\right)\log^m\left(\log\left(\frac{1}{\epsilon}\right)\right)$. If $m\sigma^+ < 1$, the sum-in-QAE method can be better than the naive iteration method. In particular, when $\sigma \leq 0$, the logarithmic factor for the naive iteration method is replaced with the doubly logarithmic factor in the sum-in-QAE method, and therefore the latter method is promising. Note that, of course, the above discussion is comparison of the asymptotic upper bounds of query numbers with constant factors omitted, and the actual best method can vary depending on the problem.

IV. SUMMARY

In this paper, we considered the quantum methods to calculate derivatives of an expected value with a parameter, that is, $V(x) = E[F(S, x)]$, where F is a function of a stochastic variable S and a real parameter x and $E[\cdot]$ denotes the expectation with respect to the randomness of S. This is related to financial derivative pricing, an important industrial application of QMCI, since calculation of sensitivities of financial derivatives falls into this problem. Since naively applying some finite difference formula to $V(x)$ leads to a poor accuracy due to the error in calculating V divided by a small difference width h, we adopted the direction that we apply the central difference formula to F and estimate its expected value. Then, given some oracles such as $O_{F,\epsilon}$, which calculates F with finite precision, and O_S, which generates a quantum state corresponding to a probability distribution S, we concretely presented two quantum methods, and evaluated their query complexities, focusing on the dependency on the error tolerance ϵ. The first method is the naive iteration method, in which we calculate the difference formula by simply iterating calls to $O_{F,\epsilon}$ for the terms in the formula, and then estimate the expected value by QAE. The second one is the sum-in-QAE, in which we 'mix' the summation of the terms into the sum over the possible values of S, and perform these sums in one QAE at one time. We saw that there are some issues on the smoothness of F and the number of qubits available. First, if F is nonsmooth with respect to x and we yet take small h, the value of the difference formula on F can be large for some (S, x), and this leads to the large complexity in QAE. Second, even if F is smooth, in order to take small h, we need calculate V with high precision, which means that we have to use many qubits for the calculation. Considering these points, we saw that either of two methods can be advantageous against the other depending on the situation. When F is smooth and we can use many qubits, the naive iteration method with the lowest order difference formula and small h is better. Conversely, if F is nonsmooth, we can use the higher order formula with h that is $\widetilde{O}(1)$ with respect to ϵ, and the sum-in-QAE method is better. Even when F is smooth, if we want to save qubits by using the higher order formula with $h = \widetilde{O}(1)$, the sum-in-QAE method is better depending on the parameter σ, which measures the smoothness of F. In any case, we can calculate the derivative of V with $\widetilde{O}\left(\frac{1}{\epsilon}\right)$ complexity, which is same as that for calculating V itself, except for logarithmic factors.

We believe that the discussion in this paper provides us with insights on the plausible situation in the future, where we want to apply quantum algorithms to complicated industrial problems consuming many qubits, but the number of qubits available is limited. For future works, we will further aim to search industrial applications of quantum algorithms to concrete problems in a practical setting.

ACKNOWLEDGMENT

This work was supported by MEXT Quantum Leap Flagship Program (MEXT Q-LEAP) Grant Number JPMXS0120319794.

Appendix A: Proof of Lemma 1

Proof. First, we consider the case where $m \geq 2$. From the definition of $a^{(m)}_{n,j}$ in (6),

$$|a^{(m)}_{n,j}| \leq \sum_{\substack{\{l_1,\ldots,l_{m-1}\}\in \\ \mathcal{P}_{m-1}([-n:n]\setminus\{0,j\})}} \frac{(n!)^2}{|jl_1\cdots l_{m-1}|} \tag{A1}$$

holds for any $j \in [-n : n] \setminus \{0\}$. Note that

$$\begin{aligned}
&\sum_{\substack{\{l_1,\ldots,l_{m-1}\}\in \\ \mathcal{P}_{m-1}([-n:n]\setminus\{0,j\})}} \frac{1}{|l_1\cdots l_{m-1}|} \\
&\leq \frac{1}{(m-1)!}\left(\frac{1}{|-n|}+\cdots+\frac{1}{|-1|}+\frac{1}{1}+\cdots+\frac{1}{n}\right)^{m-1} \\
&\leq \frac{(2n)^{m-1}}{(m-1)!}
\end{aligned}$$

Table I: The dependencies of the number of queries to $O_{F,\tilde{\epsilon}}$ on error tolerance ϵ in the proposed methods in different cases. The ones which can be better in each case are underlined.

smoothness of F	# of qubits available	the naive iteration method (n,h)	the naive iteration method query number	the sum-in-QAE method (n,h)	the sum-in-QAE method query number
nonsmooth	—	$(n_{\rm th}, h_{\rm th})$	$\frac{1}{\epsilon}\log^{m\sigma^+ +1}\left(\frac{1}{\epsilon}\right)\log^m\left(\log\left(\frac{1}{\epsilon}\right)\right)$	$(n_{\rm th}, h_{\rm th})$	$\underline{\frac{1}{\epsilon}\log^{m\sigma^+}\left(\frac{1}{\epsilon}\right)\log^m\left(\log\left(\frac{1}{\epsilon}\right)\right)}$
smooth	large	$\left(\left\lceil\frac{m}{2}\right\rceil, h_{\rm min}\right)$	$\underline{\frac{1}{\epsilon}}$	$(n_{\rm th}, h_{\rm th})$	$\frac{1}{\epsilon}\log^{m\sigma^+}\left(\frac{1}{\epsilon}\right)\log^m\left(\log\left(\frac{1}{\epsilon}\right)\right)$
	small	$(n_{\rm th}, h_{\rm th})$	$\underline{\frac{1}{\epsilon}\log\left(\frac{1}{\epsilon}\right)}$	$(n_{\rm th}, h_{\rm th})$	$\underline{\frac{1}{\epsilon}\log^{m\sigma^+}\left(\frac{1}{\epsilon}\right)\log^m\left(\log\left(\frac{1}{\epsilon}\right)\right)}$

$$\leq \frac{m^{2n}}{(m-1)!}. \tag{A2}$$

Here, the third inequality follows from $(2n)^{m-1} \leq m^{2n}$, which holds for any positive integers m and n such that $m \leq 2n$. Then, we see that

$$|a^{(m)}_{n,j}| \leq \frac{(n!)^2 m^{2n}}{|j|(m-1)!}. \tag{A3}$$

This holds also when $m = 1$, since

$$\left|a^{(1)}_{n,j}\right| = \left|\frac{\prod_{i\in[-n:n]\setminus\{0\}} i}{j}\right| = \frac{(n!)^2}{|j|}. \tag{A4}$$

Hence, combining this and (24) with the definition of $R_f(x,n,m,h)$ in (6), we have

$$\begin{aligned}
|R_f(x,n,m,h)| &\leq \frac{Mm!\,(n!)^2\,m^{2n}}{(2n+1)!(m-1)!}\sum_{j=-n}^{n}\frac{|j|^{2n}}{(n+j)!(n-j)!} \\
&\leq \frac{Mm\,(n!)^2\,n^{2n}m^{2n}}{(2n+1)!(2n)!}\sum_{j=-n}^{n}\binom{2n}{n+j} \\
&\leq \frac{Mm\,(n!)^2\,(2n)^{2n}m^{2n}}{(2n+1)!(2n)!} \\
&\leq \frac{Mm(2n)^{2n}m^{2n}}{(2n+1)}\left(\frac{n^{n+\frac{1}{2}}e^{-n}e^{\frac{1}{12n}}}{(2n)^{2n+\frac{1}{2}}e^{-2n}e^{\frac{1}{24n+1}}}\right)^2 \\
&= \frac{Mm}{2(2n+1)}\left(\frac{em}{2}\right)^{2n} e^{\frac{1}{6n}-\frac{2}{24n+1}} \\
&\leq Mm\left(\frac{em}{2}\right)^{2n}.
\end{aligned} \tag{A5}$$

Here, we used $\sum_{j=0}^{k}\binom{k}{j} = 2^k$, which holds for any $k \in \mathbb{N}$, at the third inequality, and

$$\sqrt{2\pi}n^{n+\frac{1}{2}}e^{-n}e^{\frac{1}{12n+1}} < n! < \sqrt{2\pi}n^{n+\frac{1}{2}}e^{-n}e^{\frac{1}{12n}}, \tag{A6}$$

which is given in [48], at the fourth inequality. □

Appendix B: Proof of Lemma 2

Proof. When $m \geq 2$, because of (A1), we have

$$\begin{aligned}
\left|d^{(m)}_{n,j}\right| &\leq \frac{m!(n!)^2}{(n+j)!(n-j)!|j|}\sum_{\substack{\{l_1,\dots,l_{m-1}\}\in \\ \mathcal{P}_{m-1}([-n:n]\setminus\{0,j\})}}\frac{1}{|l_1\cdots l_{m-1}|} \\
&\leq \frac{m!}{|j|}\sum_{\substack{\{l_1,\dots,l_{m-1}\}\in \\ \mathcal{P}_{m-1}([-n:n]\setminus\{0,j\})}}\frac{1}{|l_1\cdots l_{m-1}|}
\end{aligned} \tag{B1}$$

for any $j \in [-n:n]\setminus\{0\}$, where we used

$$\frac{(n!)^2}{(n+j)!(n-j)!} = \frac{n(n-1)\cdots(n-j+1)}{(n+j)(n+j-1)\cdots(n+1)} \leq 1.$$

Therefore, we obtain

$$\begin{aligned}
D^{(m)}_n &= \sum_{j=-n}^{n}\left|d^{(m)}_{n,j}\right| \\
&\leq 2\sum_{j\in[-n:n]\setminus\{0\}}\left|d^{(m)}_{n,j}\right| \\
&\leq \sum_{j\in[-n:n]\setminus\{0\}}\frac{2m!}{|j|}\sum_{\substack{\{l_1,\dots,l_{m-1}\}\in \\ \mathcal{P}_{m-1}([-n:n]\setminus\{0,j\})}}\frac{1}{|l_1\cdots l_{m-1}|} \\
&\leq 2m!m\sum_{\substack{\{l_1,\dots,l_m\}\in \\ \mathcal{P}_m([-n:n]\setminus\{0\})}}\frac{1}{|l_1\cdots l_m|} \\
&\leq 2m!m\times\frac{1}{m!}\left(\frac{1}{|-n|}+\cdots+\frac{1}{|-1|}+\frac{1}{1}+\cdots+\frac{1}{n}\right)^m \\
&\leq 2m\left[2\left(1+\log n\right)\right]^m,
\end{aligned} \tag{B2}$$

where the first equality follows from the definition of $d^{(m)}_{n,0}$ in (6).

When $m = 1$, $|a^{(1)}_{n,j}| = (n!)^2/|j|$ for $j \in [-n:n]\setminus\{0\}$, and therefore

$$\left|d^{(1)}_{n,j}\right| = \frac{(n!)^2}{(n+j)!(n-j)!|j|} \leq \frac{1}{|j|}. \tag{B3}$$

This leads to

$$\begin{aligned} D_n^{(m)} &\le 2 \sum_{j\in[-n:n]\setminus\{0\}} \left| d_{n,j}^{(m)} \right| \\ &\le 4 \sum_{j=1}^{n} \frac{1}{j} \\ &\le 4(1+\log n), \end{aligned} \tag{B4}$$

which indicates that (27) holds also for $m = 1$. □

Appendix C: Proof of Lemma 3

To prove Lemma 3, let us introduce some additional lemmas.

Lemma 4. *For any $a, x \in \mathbb{R}_+$ satisfying $a^2 \le x$,*

$$x \ge a \log x \tag{C1}$$

holds.

Proof. Consider a function $f_a(x) = x - a\log x$ on $\mathbb{R}_+$. $f_a'(x) = 1 - \frac{a}{x}$, and therefore it takes the minimum $a(1-\log a)$ at $x = a$. Then, we consider the following cases.

(i) $a \le e$

Since the minimum of $f_a(x)$ is larger than 0,

$$\forall x \in \mathbb{R}_+, f_a(x) \ge 0, \tag{C2}$$

which means (C1) holds for any $x \in \mathbb{R}_+$.

(i) $a > e$

As a special case of (C2), $f_2(y) = y - 2\log y \ge 0$ holds for any $y \in \mathbb{R}_+$, which indicates that $f_a(a^2) = a(a - 2\log a) \ge 0$. Besides, $f_a'(x) > 0$ for any x larger than a^2. Combining these, we see that, for any x larger than a^2, $f_a(x) \ge 0$ holds, and so does (C1).

□

Lemma 5. *For any $a \in \mathbb{R}_{\ge 0}$, $x \in \mathbb{R}_+$, and $\epsilon \in \mathbb{R}_+$ such that*

$$\epsilon \le \begin{cases} 2^{a-\left(\frac{a}{\log 2}\right)^2} & ;\ \text{for } a \ge \log 2 \\ 2^{a-1} & ;\ \text{for } a < \log 2 \end{cases} \tag{C3}$$

and

$$x \ge \tilde{x}_{a,\epsilon} := \log_2\left(\frac{2^a}{\epsilon}\right) + a\log_2\left(\log_2\left(\frac{2^a}{\epsilon}\right)\right), \tag{C4}$$

$$\frac{x^a}{2^x} \le \epsilon \tag{C5}$$

holds.

Proof. When $a = 0$, (C5) becomes $2^{-x} \le \epsilon$ and holds trivially, since (C4) becomes $x \ge \log_2\left(\frac{1}{\epsilon}\right)$. Therefore, we consider the case where $a > 0$ below. First, note that, under the condition (C3),

$$\log_2\left(\frac{2^a}{\epsilon}\right) \ge \left(\frac{a}{\log 2}\right)^2 \tag{C6}$$

holds, and this means

$$\log_2\left(\frac{2^a}{\epsilon}\right) \ge \frac{a}{\log 2}\log\left(\log_2\left(\frac{2^a}{\epsilon}\right)\right) = a\log_2\left(\log_2\left(\frac{2^a}{\epsilon}\right)\right), \tag{C7}$$

because of Lemma 4. Then, we see that

$$\begin{aligned} \frac{(\tilde{x}_{a,\epsilon})^a}{2^{\tilde{x}_{a,\epsilon}}} &= \frac{\left[\log_2\left(\frac{2^a}{\epsilon}\right) + a\log_2\left(\log_2\left(\frac{2^a}{\epsilon}\right)\right)\right]^a}{\frac{2^a}{\epsilon}\left(\log_2\left(\frac{2^a}{\epsilon}\right)\right)^a} \\ &\le \frac{2^a\left(\log_2\left(\frac{2^a}{\epsilon}\right)\right)^a}{\frac{2^a}{\epsilon}\left(\log_2\left(\frac{2^a}{\epsilon}\right)\right)^a} \\ &= \epsilon, \end{aligned} \tag{C8}$$

where we use the definition (C4) of $\tilde{x}_{a,\epsilon}$ at the first equality, and (C7) at the inequality.

On the other hand, defining $g(x) := \frac{x^a}{2^x}$, we obtain

$$\frac{d}{dx}\log g(x) = \frac{a}{x} - \log 2, \tag{C9}$$

which implies that, when $x \ge \frac{a}{\log 2}$, $\log g(x)$ is monotonically deceasing, and so is $g(x)$. Besides, under (C3), we can see that both

$$\log_2\left(\frac{2^a}{\epsilon}\right) \ge 1 \tag{C10}$$

and

$$\log_2\left(\frac{2^a}{\epsilon}\right) \ge \frac{a}{\log 2} \tag{C11}$$

hold by simple algebra, and therefore

$$\tilde{x}_{a,\epsilon} = \log_2\left(\frac{2^a}{\epsilon}\right) + a\log_2\left(\log_2\left(\frac{2^a}{\epsilon}\right)\right) \ge \frac{a}{\log 2}. \tag{C12}$$

Hence, we obtain

$$\frac{x^a}{2^x} \le \frac{(\tilde{x}_{a,\epsilon})^a}{2^{\tilde{x}_{a,\epsilon}}} \le \epsilon \tag{C13}$$

for $x \ge \tilde{x}_{a,\epsilon}$.

□

Now, let us prove Lemma 3.

Proof of Lemma 3. Under (28) and (30), Lemma 5 implies that

$$\frac{(2n+1)^{m\sigma^+}}{2^{2n+1}} \le \epsilon'. \tag{C14}$$

Then, we see that

$$
\begin{aligned}
&\left|R_f(x,n,m,h)h^{2n-m+1}\right| \\
&\le Ac^{2n+1}\left((2n+1)!\right)^{\sigma} m\left(\frac{em}{2}\right)^{2n} h^{2n-m+1} \\
&\le Ac^{2n+1}\left((2n+1)^{\sigma^+}\right)^{2n+1} m\left(\frac{em}{2}\right)^{2n} h^{2n-m+1} \\
&= \frac{2A}{e}\left(\frac{ecm(2n+1)^{\sigma^+}h}{2}\right)^{2n+1} h^{-m} \\
&\le \frac{2(ecm)^m A}{e}\frac{(2n+1)^{m\sigma^+}}{2^{2n+1}} \\
&\le \frac{2(ecm)^m A}{e}\epsilon' \\
&= \epsilon. \qquad \text{(C15)}
\end{aligned}
$$

Here, at the first inequality, we use (25) with $M = Ac^{2n+1}\left((2n+1)!\right)^{\sigma}$, since $f \in \mathcal{G}_{A,C,\sigma}$. We also use $\left((2n+1)!\right)^{\sigma} \le \left((2n+1)^{\sigma^+}\right)^{2n+1}$ at the second inequality, (29) at the third inequality, (C14) at the fourth inequality, and (31) at the last equality. Because of Theorem 1, (C15) indicates that (32) holds.

□

[1] A. Montanaro, "Quantum speedup of Monte Carlo methods", Proc. Roy. Soc. Ser. A, 471, 2181 (2015)

[2] Y. Suzuki et al., "Amplitude Estimation without Phase Estimation", Quantum Inf. Process. 19, 75 (2020)

[3] S. Herbert, "Quantum Monte-Carlo Integration: The Full Advantage in Minimal Circuit Depth", arXiv:2105.09100 (2021)

[4] G. Brassard et al., "Quantum amplitude amplification and estimation", Contemporary Mathematics 305, 53 (2002)

[5] S. Aaronson and P. Rall, "Quantum approximate counting, simplified", in Proccedings of Symposium on Simplicity in Algorithms, 24–32, SIAM (2020)

[6] D. Grinko et al., "Iterative quantum amplitude estimation", npj Quantum Inf. 7, 52 (2021)

[7] K. Nakaji, "Faster Amplitude Estimation", Quantum Inf. Comput. 20, 1109 (2020)

[8] E. G. Brown et al., "Quantum amplitude estimation in the presence of noise", arXiv:2006.14145

[9] I. Kerenidis and A. Prakash, "A method for amplitude estimation with noisy intermediate-scale quantum computers", U.S. Patent Application No. 16/892,229 (2020)

[10] T. Giurgica-Tiron et al., "Low depth algorithms for quantum amplitude estimation", arXiv:2012.03348

[11] T. Tanaka et al., "Amplitude estimation via maximum likelihood on noisy quantum computer", Quantum Inf. Process. 20, 293 (2021)

[12] S. Uno et al., "Modified Grover operator for amplitude estimation", New J. Phys. 23, 083031 (2021)

[13] G. Wang et al., "Minimizing Estimation Runtime on Noisy Quantum Computers", PRX Quantum 2, 010346 (2021)

[14] J. C. Hull, "Options, Futures, and Other Derivatives", Prentice Hall, Englewood Cliffs, NJ (1995)

[15] S. P. Jordan, "Fast quantum algorithm for numerical gradient estimation", Phys. Rev. Lett. 95, 050501 (2005)

[16] A. Gilyén, S. Arunachalam, and N. Wiebe, "Optimizing quantum optimization algorithms via faster quantum gradient computation", Proceedings of the 30th ACM-SIAM Symposium on Discrete Algorithms (SODA 2019), pp. 1425-1444, SIAM (2019)

[17] A. Cornelissen, "Quantum gradient estimation of Gevrey functions", arXiv:1909.13528 (2019)

[18] P. Glasserman, "Monte Carlo Methods in Financial Engineering", Springer (2003)

[19] A. G. Fowler et al., "Surface codes: Towards practical large-scale quantum computation", Phys. Rev. A 86, 032324 (2012)

[20] J. Li, "General explicit difference formulas for numerical differentiation", Journal of Computational and Applied Mathematics, 183, 29 (2005)

[21] V. Vedral, A. Barenco and A. Ekert, "Quantum Networks for Elementary Arithmetic Operations", Phys. Rev. A 54, 147 (1996)

[22] T. G. Draper, "Addition on a quantum computer", arXiv:quant-ph/0008033 (2000)

[23] S. A. Cuccaro et al., "A new quantum ripple-carry addition circuit", arXiv:quant-ph/0410184 (2004)

[24] Y. Takahashi et al., "A linear-size quantum circuit for addition with no ancillary qubits", Quantum Inf. Comput. 5, 440 (2005)

[25] T. G. Draper et al., "A logarithmic-depth quantum carry-lookahead adder", Quantum Inf. Comput. 6, 351 (2006)

[26] J. J. Alvarez-Sanchez et al., "A quantum architecture for multiplying signed integers", Journal of Physics: Conference Series 128, 012013 (2008)

[27] Y. Takahashi et al., "A fast quantum circuit for addition with few qubits", Quantum Inf. Comput. 8, 636 (2008)

[28] Y. Takahashi et al., "Quantum addition circuits and unbounded fan-out", Quantum Inf. Comput. 10, 0872 (2010)

[29] A. Khosropour et al., "Quantum division circuit based on restoring division algorithm", in Proceedings of 2011 Eighth International Conference on Information Technology: New Generations, pp. 1037-1040 (2011)

[30] H. Thapliyal and N. Ranganathan, "Design of Efficient Reversible Logic Based Binary and BCD Adder Circuits", J. Emerg. Technol. Comput. Syst. 9, 17 (2013)

[31] H. Thapliyal, "Mapping of subtractor and adder-subtractor circuits on reversible quantum gates", Transactions on Computational Science XXVII, 10 (2016)

[32] H. V. Jayashree et al., "Ancilla-input and garbage-output optimized design of a reversible quantum integer multiplier", The Journal of Supercomputing 72, 1477 (2016)

[33] S. V. Dibbo et al., "An efficient design technique of a quantum divider circuit", in Proceedings of 2016 IEEE International Symposium on Circuits and Systems (ISCAS), pp. 2102-2105 (2016)

[34] E. Muñoz-Coreas and H. Thapliyal, "T-count and Qubit Optimized Quantum Circuit Design of the Non-Restoring Square Root Algorithm", ACM Journal on Emerging Technologies in Computing Systems 14, 3 (2018)

[35] E. Muñoz-Coreas and H. Thapliyal, "Quantum Circuit Design of a T-count Optimized Integer Multiplier", IEEE Transactions on Computers 68, 5 (2019)

[36] H. Thapliyal et al., "Quantum Circuit Designs of Integer Division Optimizing T-count and T-depth", IEEE Transactions on Emerging Topics in Computing 9, 1045 (2019)

[37] T. Häner, M. Roetteler and K. M. Svore, "Optimizing Quantum Circuits for Arithmetic", arXiv:1805.12445 (2018)

[38] L. Grover and T. Rudolph, "Creating superpositions that correspond to efficiently integrable probability distributions", quant-ph/0208112 (2002)

[39] K. Kaneko et al., "Quantum Pricing with a Smile: Implementation of Local Volatility Model on Quantum Computer", arXiv:2007.01467 (2020)

[40] K. Miyamoto and K. Shiohara, "Reduction of Qubits in Quantum Algorithm for Monte Carlo Simulation by Pseudo-random Number Generator", Phys. Rev. A 102, 022424 (2020)

[41] M. Mottonen et al., "Transformation of quantum states using uniformly controlled rotations", Quant. Inf. Comp. 5, 467 (2005)

[42] V. Bergholm et al., "Quantum circuits with uniformly controlled one-qubit gates", Phys. Rev. A 71, 052330 (2005)

[43] V. V. Shende, S. S. Bullock, and I. L. Markov, "Synthesis of Quantum Logic Circuits", IEEE Trans. on Computer-Aided Design 25, 1000 (2006)

[44] M. Plesch and C. Brukner, "Quantum-state preparation with universal gate decompositions", Phys. Rev. A 83, 032302 (2011)

[45] R. Iten et al., "Quantum circuits for isometries", Phys. Rev. A 93, 032318 (2016)

[46] D. K. Park, F. Petruccione and J.-K. K. Rhee, "Circuit-based quantum random access memory for classical data", Sci. reports 9, 3949 (2019)

[47] I. S. Araujo et al., "A divide-and-conquer algorithm for quantum state preparation", Sci. Rep. 11, 6329 (2021)

[48] H. Robbins, "A Remark on Stirling's Formula", The American Mathematical Monthly, 62, 26 (1955)